

Project 2 Report

Group 9: Elliot Mai, Carl Nguyen, Noe Sanchez

Overall Status:

Read/Write Functions:

The transaction manager's *TxRead* and *TxWrite* functions serve as the entry points for read and write operations. Both follow a similar pattern: they create a parameter structure with the transaction ID, object number, and operation type ('R' for read, 'W' for write), decrement the transaction's sequence counter to maintain operation order, and spawn a thread that executes the appropriate transaction function.

The transaction class's *readtx* and *writetx* functions act as thread entry points, extracting the operation parameters and delegating to the common *process_read_write_operation* function with the appropriate mode specifier. This shared function handles the mechanics of both operations through a unified approach: it synchronizes transaction operations using mutex locks, verifies the transaction's validity, attempts to set the appropriate lock, and performs the actual data manipulation once the lock is acquired.

The *perform_read_write_operation* function executes the core data modifications following specific rules: read operations decrements the object value by 4, while write operations increments it by 7. Each operation's transaction details, object information, and timing data is logged into a file to maintain a complete audit trail of all database changes.

Commit/Abort Functions:

The transaction class implements commit and abort functionality through thread functions *committx* and *aborttx*, which extract transaction information and delegate to a common *do_commit_abort_operation* function with the appropriate status flag (TR_END for commit, TR_ABORT for abort). This shared function retrieves the transaction object, logs the operation type, releases all locks held by the transaction, signals any waiting transactions to proceed, removes the transaction from the manager's list, and marks the operation as complete.

This approach in handling commit and abort operations ensures consistent cleanup regardless of whether a transaction completes successfully or terminates prematurely, maintaining database integrity in all scenarios.

Set Lock Function:

The *set_lock* function forms the backbone of the concurrency control mechanism. It implements a strict two-phase locking protocol that ensures serializability of transactions. The function first validates that the transaction exists and is in a valid state to acquire locks, then checks if the requested object is already locked by consulting the lock hash table.

If the object is available, the function records the lock acquisition in the hash table and sets the transaction status to active. If the lockmode is different from the object present in the hash table, another object node with the different lockmode and same transaction is added to the

hash table. Else, another object is not added. However, if another transaction holds the lock, it sets the current transaction's status to waiting, associates it with the lock holder's semaphore, and blocks the current running thread until signaled when the lock becomes available.

The coordination of lock acquisition and release along with the transaction status management, prevents conflicts and ensures that concurrent transactions execute in a serializable manner, maintaining the ACID properties essential to database integrity.

Difficulties:

It was slightly difficult to understand what certain functions were for inside `zgt_tx.c`. For example, there was both a `process_read_write_operation()` and `perform_read_write_function()` present in `zgt_tx.c` that were both left for the team to fill out to handle read and write operations, but the team was a bit unsure about the differences between them or their exact necessity since it seemed possible to combine them.

A small difficulty also occurred with understanding the use of the `findt()` function provided. In `zgt_tx.h`, it was stated in a comment as “find the tx obj belongs to” but in `zgt_ht.c` where it was defined, it was stated to “return the object held by a Tx; else null”. It took a bit to realize that it returns an object node if it contained the same tid of the transaction, which could then be used to determine which transaction the object belonged to.

Another difficulty was understanding how running the code would be like. The team was uncertain whether certain aspects such as duration of testing with certain files or certain output should be occurring. More explanation of what running the code should be like or sample run outputs would be helpful at determining issues with the code the team needs to fix.

The semaphore refresher was very helpful though for understanding how to apply semaphores and understanding how `zgt_p()` and `zgt_w()` worked.

Additional Notes:

Certain runs resulted in segfaults. When checking the log file, it seems that the transaction number was a value not given within the file for a begin operation. For example, instead of receiving and creating transaction T1, it would sometimes randomly receive and create transaction T15. Since the team did not want to mess with the already pre existing code and due to existing time constraints, the team was unable to figure out what was causing the issue.

The code would at times also hang if the end operation occurred after the last transaction manages to commit or abort regardless if other transactions were currently waiting and set free, resulting in a hang.

File Descriptions:

Test Files:

RR3.txt:

This test case tests if multiple transactions can read the same object concurrently without conflicts while maintaining correct transaction sequencing.

MRW3.txt:

This test case tests if multiple transactions correctly handle locks while reading and writing shared objects. Transaction 1 reads and writes objects 1, 2, and 3, while transaction 2 reads objects 2 and 3, testing shared read access. Transaction 3 writes to object 1 and reads 3, checking if locks are properly enforced to prevent conflicts. The goal is to validate the Strict Two-Phase Locking (2PL) mechanism by ensuring transactions acquire and release locks correctly while preventing dirty reads, lost updates, and inconsistent retrievals.

Read10.txt:

This test case consists of 10 transactions, each performing a single read operation on a different object. The purpose of this test is to verify that the system correctly handles concurrent read-only transactions without introducing conflicts and could handle the max number of transactions. Since these are all read operations, they should not block or interfere with each other.

Division of Labor:

The Division of Work:

Job	Assigned.
Implement <i>zgt_tm::TxWrite</i>	Noe
Implement <i>zgt_tm::CommitTx</i>	Noe
Implement <i>zgt_tm::AbortTx</i>	Noe
Implement <i>*readtx</i>	Noe
Implement <i>*writetx</i>	Noe
Implement <i>zgt_tx::perform_read_write_operation</i>	Noe
Implement <i>zgt_tx::process_read_write_operation</i>	Noe
Implement <i>*aborttx</i>	Elliot & Carl
Implement <i>*committx</i>	Elliot & Carl
Implement <i>*do_commit_abort_operation</i>	Elliot & Carl
Implement <i>zgt_tx::set_lock</i>	Elliot & Carl
Brainstorm Implementation	All Members
Research Documentation	All Members
Testing and Debugging	All Members

The Division of Time:

Member	Time (in Hours)
Elliot M.	15
Carl N.	16
Noe S.	17

Logical Errors and Resolutions:

Logical Error 1: Not changing tx_status from TR_WAIT:

In `zgt_tx::set_lock`, the team was correctly setting the transaction's status to `TR_WAIT` if another object held the required lock, but the status was never updated to `TR_ACTIVE` later on when the waiting transaction finally received the lock. This error was discovered when the logfiles were showing transactions as waiting when they should have been active, which was then disrupting the execution of transactions and causing excessive hanging of threads. The team resolved this issue by ensuring to set the transaction status to `TR_ACTIVE` once it acquired the lock.

Logical Error 2: Updating incorrect semaphore number:

When trying to block a waiting operation thread on a semaphore or when trying to lock objects for a specific transaction, incorrect semaphore numbers were initially used. Semaphore numbers for all transactions were initialized to -1 and were not yet updated before calling the `p` operation with it using `zgt_p()`. As a result, waiting threads would wait for the wrong semaphore, and there was the possibility of the incorrect transaction operation performing next due to believing the object it is looking for is available when it wasn't after the `v` operation (`zgt_v()`) was called with said semaphore number.

To fix this issue, the team decided to set the semaphore number of the lock-holding transaction to its `tid` value (after casting it as an `int`) using the `setTx_semno()` function to ensure that it was possible to recognize that other transaction operations were specifically waiting for a certain transaction to finish and free its hold on all the required object locks. This would be done before calling `zgt_p()` on the updated semaphore number. The semaphore number would be reseted back to -1 again later on when aborting/committing the transaction to indicate that no other transaction operations needed to wait on the certain transaction anymore once all locks that the transaction had were freed. (Changing the transaction's semaphore value back to -1 is not required since the transaction is removed from the transaction manager after the commit/abort, but was done as a safety precaution.)

Logical Error 3, Object Value Modification:

A logical error was encountered while attempting to set the `obno` value to -4 and 7. This error resulted in incorrect value updates, which were resolved by modifying/updating the existing value appropriately rather than simply overwriting it. Initially, direct assignments of `value = -4` and `value = 7` were made, with the expectation that the object's value would update accordingly. However, this approach proved ineffective, as the modification needed to be applied to `ZGT_Sh->objarray[objno]->value`. Instead of direct assignment, the correct method was to adjust the value using `--4` for the read operation and `+= 7` for the write operation.